



RV College of Engineering®

Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

PyVulSev: A Benchmark for Adversarial Robustness in Python Vulnerability Detection

MAJOR PROJECT REPORT (TODO Subject Code)

Submitted by,

TODO Your Full Name

TODO Your USN

Under the guidance of

TODO Internal Guide Name

TODO Designation

Dept. of CSE

RV College of Engineering

In partial fulfillment for the award of degree of

Bachelor of Engineering

in

Computer Science and Engineering

Department of CSE

2025-26





RV College of Engineering®

Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

Go, change the world

PyVulSev: A Benchmark for Adversarial Robustness in Python Vulnerability Detection

A Major Project Report (TODO Subject Code)

Submitted by,

TODO Your Full Name

TODO Your USN

Under the guidance of

TODO Internal Guide Name

TODO Designation

Dept. of CSE

RV College of Engineering

In partial fulfillment of the requirements for the degree of

Bachelor of Engineering in

Computer Science and Engineering

2025-26

RV College of Engineering[®], Bengaluru
(Autonomous institution affiliated to VTU, Belagavi)
Department of Computer Science and Engineering



CERTIFICATE

Certified that the major project (TODO Subject Code) work titled ***PyVulSev: A Benchmark for Adversarial Robustness in Python Vulnerability Detection*** is carried out by **TODO Your Full Name (TODO Your USN)** who is bonafide student of RV College of Engineering, Bengaluru, in partial fulfillment of the requirements for the degree of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belagavi during the year 2025-26. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the major project report deposited in the departmental library. The major project report has been approved as it satisfies the academic requirements in respect of major project work prescribed by the institution for the said degree.

Project Guide	Head of the Department	Dean CS Cluster	Principal
TODO Internal Guide Name	Dr. Shanta Rangaswamy	Dr. Ramakanth Kumar P	Dr. K. N. Subramanya

External Viva Examination

Name of Examiner

Signature with Date

- 1.
- 2.

DECLARATION

I, **TODO Your Full Name** student of eighth semester B.E., Department of Computer Science and Engineering, RV College of Engineering, Bengaluru-560059, bearing USN: **TODO Your USN**, hereby declare that the major project titled '**PyVulSev: A Benchmark for Adversarial Robustness in Python Vulnerability Detection**' has been carried out by me and submitted in partial fulfilment for the award of degree of **Bachelor of Engineering in Computer Science and Engineering** during the year 2025-26.

Further I declare that the content of the dissertation has not been submitted previously by anybody for the award of any degree or diploma to any other university.

I also declare that any Intellectual Property Rights generated out of this project carried out at RVCE will be the property of RV College of Engineering, Bengaluru and I will be one of the authors of the same.

Place: Bengaluru

Date:

Name

Signature

1. TODO Your Full Name(TODO Your USN)

ACKNOWLEDGEMENT

I am indebted to my guide, **TODO Internal Guide Name**, TODO Designation, Department of CSE, RVCE for the wholehearted support, suggestions and invaluable advice throughout my project work and also helped in the preparation of this thesis.

I also express our gratitude to my panel members **TODO Panel Member A**, TODO Designation, Department of CSE, RVCE and **TODO Panel Member B**, TODO Designation, Department of CSE, RVCE for the valuable comments and suggestions during the phase evaluations.

My sincere thanks to the project coordinators **TODO Project Co-ordinator A**, TODO Designation, Department of CSE and **TODO Project Co-ordinator B**, TODO Designation, for their timely instructions and support in coordinating the project.

My gratitude to **Prof. Narashimaraja P**, Department of ECE, RVCE for the organized latex template which made report writing easy and interesting.

My sincere thanks to **Dr. Shanta Rangaswamy**, Professor and Head, Department of Computer Science and Engineering, RVCE and **Dr. Ramakanth Kumar P**, Professor and Dean (CSE Cluster), RVCE for the support and encouragement.

I express sincere gratitude to our beloved **Dr. M.V. Renukadevi**, Professor and Dean Academics, RVCE, **Dr. Geetha K S**, Professor and Vice Principal, RVCE and **Dr. K. N. Subramanya**, Principal, RVCE for the appreciation towards this project work.

I thank all the teaching staff and technical staff of Computer Science and Engineering department, RVCE for their help. Lastly, I take this opportunity to thank my family members and friends who provided all the backup support throughout the project work.

TODO Your Full Name
Computer Science and Engineering,

RV College of Engineering,
Bengaluru-59.



ABSTRACT

Abstract is essentially a complete overview or the Synopsis of the project in one page, should be precise, concise, and clear. It has to be written in past tense as the work is already completed. If abbreviations or acronyms need to be included in the abstract, the first time it is used, it should be written out in the full form and put the abbreviation in brackets. e.g. Magnetic Resonance Imaging (MRI). The abstract may have about 3 paragraphs covering the outline of the work as below:

First para: Brief introduction about the project which should include background, Areas/type of applications, market size (if available) and hence the importance of the domain. Key developments in the domain, unresolved issues or/and new emerging opportunities in the domain areas and hence the problem or the issue that is proposed to be addressed in the project or the objective. If it is theory or simulation based add all the existing theory/algorithms names and what is it that the project is based on, their essentially advantage or drawbacks in brief and hence the motivation to work on the project.

Second Para: Objectives, scope, Methodology of your project including the key theory, design tools or software tools or / and the experimental tools that will be used, the key specifications including constraints or boundary conditions or approximations or assumptions that have been considered for the work. The sequence in which the work has been carried out.

Third para: Key findings, results or outcome of the work including experimental data of the out come or / and percentage in terms of working efficiency of the developed product or system.

In brief the Abstract should be

- Complete — it covers all aspects of the project
- Concise — it contains no excess wordiness or unnecessary information.
- Clear — it is readable, well organized, and not too jargon-laden.
- Cohesive — it flows smoothly between the parts.

LIST OF PUBLICATIONS

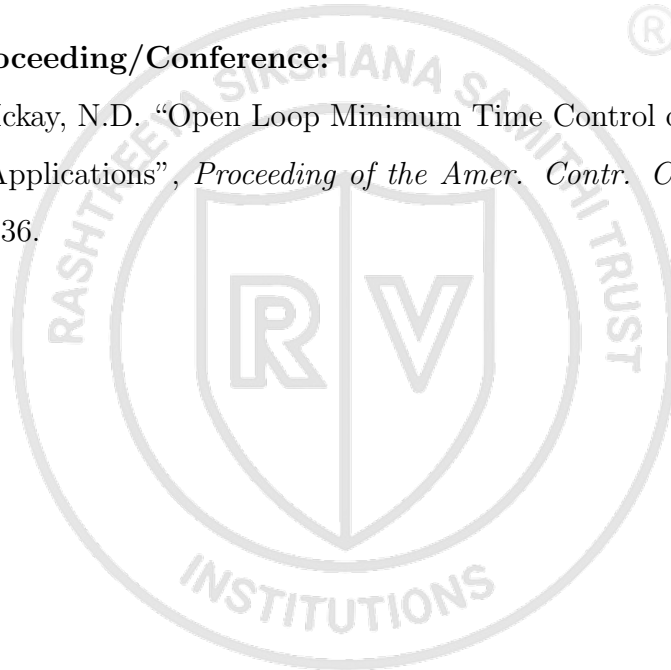
Publication Type	Publication Details
Journal / International Conference / National Conference	Format as given below for Journal (a) and Conference (b)

Example Journal Paper:

Honig, M.L., Steiglitz, K., and Gopinath, B., “Multichannel signal processing for data communication in the presence of crosstalk”, *IEEE Trans. Communications*, vol. 38, (4), 1990, pp. 551–558.

Example Proceeding/Conference:

Shin, K.G. and McKay, N.D. “Open Loop Minimum Time Control of Mechanical Manipulations and its Applications”, *Proceeding of the Amer. Contr. Conf.*, San Diego, CA, 1984, pp. 1231-1236.



CONTENTS

Abstract	i
List of Publications	ii
List of Figures	vii
List of Tables	viii
1 Introduction	1
1.1 Overview of Project Topic	2
1.1.1 Big Data Mining Platform	2
1.1.2 Global Scenario	2
1.1.3 Societal Relevance	2
1.1.4 Problem Area	3
1.2 Specific Details of the Project Topic	3
1.3 Literature Review	3
1.3.1 Sample Review	3
1.4 Problem Statement	4
1.5 Objectives	4
1.6 Scope	4
1.7 Methodology	5
1.8 Organization of the Report	5
2 Theory & Concept	6
2.1 System Architecture	7
3 Software Requirement Specification	8
3.1 Overall Description	9
3.1.1 Product Perspective	9
3.1.2 Product Functions	9
3.1.3 Constraints	10
3.2 Specific Requirements	11

3.2.1	Functional Requirements	11
3.2.2	Performance Requirements	11
3.2.3	Software Requirements	12
3.2.4	Hardware Requirements	12
3.2.5	Design Constraints	12
3.3	Summary	12
4	High Level Design	13
4.1	Design Considerations	14
4.1.1	General Considerations	14
4.1.2	Development Methods	14
4.2	Architecture Strategies	15
4.2.1	Programming Language	15
4.2.2	Future Plans	16
4.3	System Architecture	16
4.4	Data Flow Diagrams	16
4.4.1	Data Flow Diagram Level 0	17
4.4.2	Data Flow Diagram Level 1	17
4.4.3	Data Flow Diagram Level 2	17
4.5	Summary	18
5	Detailed Design	19
5.1	Structure Chart	20
5.2	Module Description	21
5.2.1	Mapper	21
5.2.2	Reducer	21
5.2.3	Selection of Centroids	22
5.3	Summary	23
6	Implementation	24
6.1	Programming Language Selection	25
6.2	Platform Selection	25
6.3	Code Conventions	26
6.3.1	Naming Convention	27

6.4	Difficulties Encountered and Strategies Used to Tackle	27
6.5	Data Set	28
6.6	Algorithms Used	28
6.6.1	Standard K-Means Algorithm	28
6.6.2	Mapper Part of K-Means Algorithm	29
6.6.3	Reducer Part of K-Means Algorithm	29
6.6.4	Selection of Centroids	30
6.7	Summary	30
7	Software Testing	31
7.1	Hadoop Cluster Testing	32
7.1.1	Test for Single Node	32
7.1.2	Test for Multi Node	33
7.2	System Testing	33
7.2.1	Test for Mapper	34
7.2.2	Test for Reducer	34
7.2.3	Test for Input Data Storage	35
7.2.4	Test for Output Data Storage	35
7.2.5	Test for Output	35
7.2.6	Test for JobTracker	36
7.2.7	Test for Number of Mappers	36
7.3	Summary	36
8	Experimental Results	38
8.1	Results Analysis	39
8.2	Detailed Analysis	40
8.2.1	Performance Analysis	40
8.2.2	Evaluation Metric	40
8.3	Summary	41
9	Conclusion	42
9.1	Limitations	43
9.2	Future Enhancement	43

A	Plagiarism report	45
B	Publication details	47
C	Code	49
C.1	First Appendix	50
	Bibliography	51



LIST OF FIGURES

1.1	Sample system architecture diagram	3
1.2	Sample methodology flow diagram	5
2.1	Hadoop Basic Architecture	7
3.1	Sample product perspective diagram	10
4.1	Sample software development methodology	15
4.2	Sample system architecture diagram	17
4.3	Sample Data Flow Diagram Level 0	17
4.4	Sample Data Flow Diagram Level 1	18
4.5	Sample Data Flow Diagram Level 2	18
5.1	Sample Structure Chart	20
5.2	Sample Mapper Module Workflow	22
5.3	Sample Reducer Module Workflow	22
5.4	Sample Centroid Selection Process	23
6.1	Sample Platform Architecture	26
6.2	Sample K-Means Algorithm Workflow	29
6.3	Sample Reducer Workflow	30
7.1	Sample Single Node Hadoop Testing	33
7.2	Sample Multi Node Hadoop Testing	34
7.3	Sample Performance Analysis for Number of Mappers	37
8.1	Sample Result Analysis	39
8.2	Sample Performance Analysis	41

LIST OF TABLES

6.1	Sample Dataset Attributes	28
7.1	Sample Mapper Test Case	34
8.1	Sample Evaluation Metrics	41





Chapter 1

Introduction

CHAPTER 1

INTRODUCTION

The rapid advancement of technology and intelligent computational systems has created significant opportunities for solving complex real-world problems efficiently. Modern applications increasingly rely on data-driven approaches, automation, and scalable architectures to improve performance and decision-making. This chapter introduces the selected project topic, discusses its global and societal relevance, and highlights the specific problem area addressed in the project. The chapter also presents the objectives, scope, methodology, and organization of the report to provide a structured understanding of the proposed work.

1.1 Overview of Project Topic

This section provides an overview of the selected project domain and introduces the fundamental concepts related to the work. The relevance of the topic, its recent developments, and its practical importance can be discussed here. Ensure to cite relevant literature wherever necessary [1]. Each figure and table must be referred to in the text before it appears.

1.1.1 Big Data Mining Platform

Detailed explanation of the data mining platform, analytical tools, and computational frameworks used in the project can be presented in this subsection. The discussion should align completely to the left and maintain proper formatting consistency throughout the report.

1.1.2 Global Scenario

Discuss the current global trends, industrial adoption, and worldwide research advancements related to the project topic. Statistical reports, case studies, or technological developments can also be included.

1.1.3 Societal Relevance

Explain the societal impact and practical importance of the proposed system. Discuss how the project contributes to improving efficiency, accessibility, sustainability, or quality of life depending on the application domain.

1.1.4 Problem Area

Describe the exact problem domain addressed by the project. Highlight the limitations of existing systems and identify the research gaps or technical challenges that motivate the proposed work.

1.2 Specific Details of the Project Topic

This section provides detailed information regarding the proposed project, including the technologies, tools, methodologies, and system functionalities involved. Ensure that abbreviations such as Magnetic Resonance Imaging (MRI) are expanded during their first occurrence in the report.

Figures, architecture diagrams, or workflow representations may be included wherever necessary.



Figure 1.1: Sample system architecture diagram

1.3 Literature Review

A literature review presents a study of previously published research works related to the selected topic. This section should summarize and critically analyze existing methodologies, techniques, and findings from journals, conference papers, books, and technical reports.

A minimum of 10 standard research papers should be reviewed in this section. Avoid using papers from predatory journals or unreliable sources. The review should identify research gaps and justify the necessity of the proposed project.

1.3.1 Sample Review

The work proposed by various researchers focused on improving system efficiency using advanced computational techniques. Existing methods achieved promising performance;

however, limitations such as scalability, computational overhead, and reduced accuracy under complex conditions still exist.

Plagiarism

Using someone else's ideas, figures, or exact words without proper acknowledgement is considered plagiarism. All sources must be appropriately cited using standard referencing formats.

Adding References

Use JabRef or BibTeX for managing references. References can be cited using commands such as:

```
\cite{reference_key}
```

A sample citation is shown here [1].

1.4 Problem Statement

Clearly define the problem addressed by the project in a concise paragraph. This section should explain the limitations of existing systems and the need for the proposed solution.

1.5 Objectives

The objectives of the project are:

1. To analyze the limitations of existing systems in the selected problem domain.
2. To design and develop an efficient solution using suitable methodologies and technologies.
3. To evaluate the performance and effectiveness of the proposed system.

1.6 Scope

This section describes the applicability, limitations, and future enhancement possibilities of the proposed project. Mention the environments and operational conditions under which the system is intended to function.

1.7 Methodology

Discuss the methodology adopted for the project in brief. The methodology may include requirement analysis, system design, implementation, testing, and performance evaluation.

A flowchart or block diagram can also be included in this section.



Figure 1.2: Sample methodology flow diagram

1.8 Organization of the Report

This report is organized as follows:

- Chapter 2 discusses the theory and concepts related to the project domain and provides the necessary background knowledge required for understanding the proposed work.
- Chapter 3 presents the Software Requirements Specification (SRS), including functional requirements, non-functional requirements, system constraints, and software specifications.
- Chapter 4 discusses the High Level Design (HLD) of the proposed system, including architecture diagrams, module descriptions, workflows, and design considerations.



Chapter 2

Theory & Concept

CHAPTER 2

THEORY & CONCEPT

2.1 System Architecture

Provide theoretical background here. Avoid cutting and pasting figures from other sources. Reproduce your own using appropriate graphics software. If inevitable, cite the primary source [1].

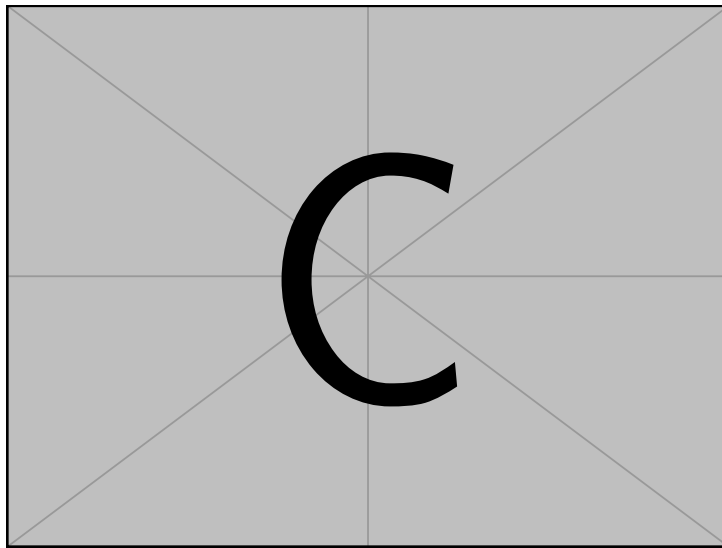


Figure 2.1: Hadoop Basic Architecture

As seen in Fig. 2.1, the basic architecture...

Chapter 3

Software Requirement Specification

CHAPTER 3

SOFTWARE REQUIREMENT SPECIFICATION

A well-defined Software Requirement Specification (SRS) provides a detailed description of the functional and non-functional requirements of the proposed system. It acts as a foundation for the design and implementation phases by clearly specifying the expected behavior, constraints, and operational requirements of the software. This chapter presents the overall description of the system, product perspective, product functions, constraints, and specific requirements including functional, performance, software, and hardware requirements. The chapter concludes with a summary of the key specifications considered during the development of the project.

3.1 Overall Description

This section provides a general description of the proposed system, its objectives, operational environment, and major functionalities. The overall system architecture and its interaction with users or external systems can also be briefly discussed here.

3.1.1 Product Perspective

Describe how the proposed system fits into the broader application domain. Explain whether the system is a standalone application, a web-based platform, a mobile application, or part of a larger integrated system.

The following aspects may also be included:

- System architecture overview
- Relationship with external systems
- User interaction with the system
- Existing system comparison

A system architecture diagram may be included as shown below.

3.1.2 Product Functions

List and briefly explain the major functionalities provided by the system. These functions should clearly describe the primary operations performed by the proposed application.



Figure 3.1: Sample product perspective diagram

Example functionalities may include:

- User authentication and authorization
- Data collection and preprocessing
- Analytics and prediction modules
- Report generation
- Dashboard visualization
- Notification and alert mechanisms

3.1.3 Constraints

Discuss the limitations and constraints under which the system is expected to operate. Constraints may include technical, operational, hardware, software, security, or resource-related limitations.

Examples include:

- Limited computational resources
- Internet dependency
- Storage limitations
- Security and privacy restrictions
- Compatibility requirements

3.2 Specific Requirements

This section provides the detailed requirements necessary for the successful implementation and operation of the proposed system. Functional and non-functional requirements should be clearly specified to ensure proper system development and validation.

3.2.1 Functional Requirements

Functional requirements define the core functionalities and services provided by the system. These requirements describe what the system should do under specific conditions.

The functional requirements may include:

1. The system shall allow users to register and log in securely.
2. The system shall process and analyze input data efficiently.
3. The system shall generate reports and visualizations based on processed data.
4. The system shall store and retrieve information from the database.
5. The system shall provide notifications or alerts when necessary.

3.2.2 Performance Requirements

Performance requirements specify the expected efficiency and responsiveness of the system during execution.

This section may include:

- System response time
- Throughput and processing speed
- Scalability requirements
- Reliability and availability
- Concurrent user handling capacity

Example:

- The system should respond to user requests within 2 seconds under normal operating conditions.
- The application should support multiple users simultaneously without significant performance degradation.

3.2.3 Software Requirements

Specify the software tools, frameworks, libraries, operating systems, and development environments required for implementing the project.

3.2.4 Hardware Requirements

Specify the minimum and recommended hardware requirements needed for developing and executing the project.

3.2.5 Design Constraints

Discuss the constraints and design limitations considered during the development of the system. These may include standards compliance, security policies, scalability considerations, or compatibility requirements.

Examples include:

- The system must comply with data privacy and security standards.
- The application should support cross-platform compatibility.
- The design should ensure scalability for future enhancements.
- The system should operate efficiently under limited hardware resources.

3.3 Summary

This chapter presented the Software Requirement Specification (SRS) for the proposed system. The overall system description, product functions, constraints, and detailed functional and non-functional requirements were discussed. The software and hardware requirements necessary for implementation were also identified. These specifications provide a strong foundation for the design and development phases discussed in the subsequent chapters.



Chapter 4

High Level Design

CHAPTER 4

HIGH LEVEL DESIGN

The High Level Design (HLD) phase provides a structured representation of the overall architecture and design considerations of the proposed system. It defines the major system components, their interactions, development methodologies, and architectural strategies adopted during implementation. This chapter presents the design considerations, programming approaches, system architecture, and data flow diagrams that collectively describe the operational workflow of the project. The chapter also highlights future enhancements and concludes with a summary of the overall design framework.

4.1 Design Considerations

This section discusses the important factors considered during the design and development of the proposed system. The design considerations ensure that the system remains scalable, reliable, maintainable, and efficient under different operating conditions.

4.1.1 General Considerations

General considerations include the overall objectives, usability requirements, system flexibility, security aspects, and performance expectations considered during the design phase.

The following factors may be discussed:

- Scalability and maintainability
- Security and privacy considerations
- User-friendly interface design
- Performance optimization
- Reliability and fault tolerance

4.1.2 Development Methods

Describe the development methodology adopted for implementing the project. Explain whether Agile, Waterfall, Iterative, Prototype, or any hybrid development model has been used.

This section may also include:

- Requirement analysis approach
- Module-wise development strategy
- Testing and validation methods
- Deployment considerations

A development lifecycle diagram may be included as shown below.



Figure 4.1: Sample software development methodology

4.2 Architecture Strategies

This section describes the architectural strategies and technologies selected for the implementation of the proposed system. The architecture should ensure efficient communication between system modules while maintaining flexibility and scalability.

4.2.1 Programming Language

Discuss the programming languages, frameworks, libraries, and tools selected for the project implementation. Justify the choice of technologies based on project requirements.

Example technologies may include:

- Python for backend processing and machine learning
- JavaScript for frontend development
- Flask/Django for web application development
- SQL or NoSQL databases for data storage

4.2.2 Future Plans

Describe the future enhancement possibilities and scalability options for the proposed system. Mention additional features, integrations, or optimizations that can be implemented in future versions.

Examples may include:

- Cloud deployment and scalability
- Mobile application integration
- Real-time analytics support
- Enhanced security mechanisms
- AI-driven automation and optimization

4.3 System Architecture

This section presents the overall architecture of the proposed system and explains the interaction between different modules and components. The architecture diagram should clearly illustrate the workflow and communication between subsystems.

The system architecture may include:

- User interface layer
- Application logic layer
- Database layer
- External APIs or services
- Machine learning or analytics modules

Figure 4.2 shows a sample system architecture diagram.

4.4 Data Flow Diagrams

Data Flow Diagrams (DFDs) represent the movement of data within the system and illustrate how information flows between different processes, entities, and storage components. DFDs help in understanding the logical functioning of the proposed system.



Figure 4.2: Sample system architecture diagram

4.4.1 Data Flow Diagram Level 0

The Level 0 Data Flow Diagram, also known as the context diagram, provides a high-level overview of the entire system. It represents the interaction between the system and external entities.

Figure 4.3 shows a sample Level 0 DFD.



Figure 4.3: Sample Data Flow Diagram Level 0

4.4.2 Data Flow Diagram Level 1

The Level 1 DFD provides a more detailed representation of the internal processes and data flow within the system. It decomposes the major processes identified in Level 0 into sub-processes.

Figure 4.4 shows a sample Level 1 DFD.

4.4.3 Data Flow Diagram Level 2

The Level 2 DFD further expands the Level 1 processes into detailed sub-processes and provides a deeper understanding of the internal system workflow.

Figure 4.5 shows a sample Level 2 DFD.



Figure 4.4: Sample Data Flow Diagram Level 1



Figure 4.5: Sample Data Flow Diagram Level 2

4.5 Summary

This chapter presented the High Level Design (HLD) of the proposed system. The design considerations, development methodologies, architectural strategies, and system architecture were discussed in detail. Data Flow Diagrams representing different levels of system abstraction were also presented to explain the flow of information within the system. The concepts discussed in this chapter provide a strong foundation for the implementation phase described in the subsequent chapters.



Chapter 5

Detailed Design

CHAPTER 5

DETAILED DESIGN

The Detailed Design phase provides an in-depth description of the internal structure, modules, and operational workflow of the proposed system. It focuses on the decomposition of the system into smaller functional components and explains how each module contributes to achieving the overall objectives of the project. This chapter presents the structure chart, module descriptions, and the detailed functionality of individual components involved in the implementation. The chapter concludes with a summary of the design specifications considered during development.

5.1 Structure Chart

A structure chart represents the hierarchical organization of modules within the proposed system. It illustrates the interaction and communication between different functional components and provides a clear understanding of the system workflow.

The structure chart may include:

- Main system modules
- Sub-modules and their relationships
- Data flow between modules
- Control hierarchy

Figure 5.1 shows a sample structure chart of the proposed system.



Figure 5.1: Sample Structure Chart

5.2 Module Description

This section provides a detailed explanation of the individual modules present in the system. Each module should be described with its functionality, inputs, outputs, and role within the overall architecture.

The module descriptions may include:

- Purpose of the module
- Functional workflow
- Input and output specifications
- Algorithms or techniques used
- Interaction with other modules

5.2.1 Mapper

The Mapper module is responsible for processing input data and transforming it into intermediate key-value pairs suitable for further computation. This module performs data partitioning, preprocessing, and mapping operations required for distributed processing or analytical computation.

The Mapper module may include:

- Data parsing and preprocessing
- Key-value pair generation
- Data filtering and transformation
- Intermediate result generation

A workflow diagram for the Mapper module may be included as shown below.

5.2.2 Reducer

The Reducer module processes the intermediate outputs generated by the Mapper module and performs aggregation, summarization, or analytical operations to produce the final result.

The Reducer module may include:



Figure 5.2: Sample Mapper Module Workflow

- Data aggregation
- Result computation
- Filtering and optimization
- Final output generation

Figure 5.3 shows a sample Reducer module workflow.



Figure 5.3: Sample Reducer Module Workflow

5.2.3 Selection of Centroids

This module explains the process of selecting centroids during clustering or analytical operations. The centroid selection process plays a crucial role in determining the efficiency and accuracy of clustering algorithms such as K-Means.

The discussion may include:

- Initial centroid selection methods
- Distance calculation techniques

- Cluster assignment process
- Iterative centroid updates
- Convergence conditions

A sample centroid selection workflow can be represented as shown below.



Figure 5.4: Sample Centroid Selection Process

5.3 Summary

This chapter presented the Detailed Design of the proposed system, including the structure chart and comprehensive descriptions of the major modules involved in the implementation. The functionalities of the Mapper, Reducer, and centroid selection modules were discussed along with their operational workflows and interactions. The detailed design presented in this chapter serves as the foundation for the implementation and testing phases discussed in the subsequent chapters.



Chapter 6

Implementation

CHAPTER 6

IMPLEMENTATION

The implementation phase focuses on transforming the proposed design into a functional system using suitable programming languages, platforms, tools, and algorithms. This chapter presents the technologies selected for development, coding standards followed during implementation, challenges encountered during development, and the strategies adopted to overcome them. The chapter also discusses the dataset used for experimentation and explains the algorithms implemented within the system. The implementation details presented in this chapter provide a clear understanding of the practical realization of the proposed project.

6.1 Programming Language Selection

This section discusses the programming languages selected for implementing the proposed system. The choice of programming language should be justified based on factors such as performance, scalability, compatibility, ease of development, and library support.

The discussion may include:

- Features of the selected programming language
- Advantages for the proposed application
- Frameworks and libraries used
- Compatibility with system requirements

Example technologies may include:

- Python for machine learning and backend processing
- JavaScript for frontend development
- SQL for database management

6.2 Platform Selection

This section describes the platforms, tools, operating systems, and development environments used during the implementation phase.

The platform selection may include:

- Operating system used for development
- Integrated Development Environment (IDE)
- Cloud or local deployment environment
- Database platforms
- APIs and external services

A deployment or platform architecture diagram may also be included.



Figure 6.1: Sample Platform Architecture

6.3 Code Conventions

Code conventions are essential for maintaining readability, consistency, scalability, and maintainability of the software. This section explains the coding standards and best practices followed during implementation.

The conventions may include:

- Indentation and formatting standards
- Commenting practices
- File and folder organization
- Error handling mechanisms
- Documentation standards

6.3.1 Naming Convention

This subsection explains the naming standards followed for variables, functions, classes, modules, files, and database entities during implementation.

Examples include:

- Variables written in camelCase or snake_case
- Class names written in PascalCase
- Meaningful and descriptive function names
- Consistent naming for database tables and APIs

Sample code snippets may also be included.

```
def calculate_aqi():  
    pass
```

6.4 Difficulties Encountered and Strategies Used to Tackle

This section discusses the technical and operational challenges faced during implementation and the strategies adopted to overcome them.

The difficulties may include:

- Dataset preprocessing challenges
- Integration issues between modules
- Performance bottlenecks
- Hardware or software compatibility issues
- Scalability and optimization problems

The corresponding solutions and mitigation strategies should also be clearly explained.

6.5 Data Set

Describe the dataset used for the implementation and evaluation of the project. Mention the source, format, size, preprocessing techniques, and relevant attributes of the dataset.

This section may include:

- Data collection methods
- Number of records and features
- Data cleaning and preprocessing
- Handling of missing values
- Data visualization and analysis

A sample dataset representation may be included.

Table 6.1: Sample Dataset Attributes

Attribute	Type	Description
ID	Integer	Unique Identifier
Value	Float	Measured Parameter
Timestamp	DateTime	Time of Observation

6.6 Algorithms Used

This section explains the algorithms implemented in the proposed system. Discuss the workflow, mathematical concepts, advantages, and implementation details of the selected algorithms.

The algorithms should be explained with proper flowcharts, pseudo-code, or diagrams wherever necessary.

6.6.1 Standard K-Means Algorithm

The K-Means algorithm is a clustering technique used to partition data into distinct clusters based on similarity. It iteratively assigns data points to clusters and updates centroid positions until convergence is achieved.

The steps involved in the K-Means algorithm include:

1. Initialize cluster centroids

2. Assign data points to nearest centroids
3. Update centroid positions
4. Repeat until convergence

Figure 6.2 shows the workflow of the standard K-Means algorithm.



Figure 6.2: Sample K-Means Algorithm Workflow

6.6.2 Mapper Part of K-Means Algorithm

This subsection explains the Mapper phase of the K-Means algorithm in a distributed processing environment. The Mapper processes input data and generates intermediate key-value pairs based on cluster assignments.

The Mapper functionality may include:

- Reading input data
- Calculating distances from centroids
- Assigning clusters
- Generating intermediate outputs

6.6.3 Reducer Part of K-Means Algorithm

The Reducer phase processes the intermediate outputs generated by the Mapper and computes updated centroid values for each cluster.

The Reducer functionality may include:

- Aggregating cluster data
- Calculating updated centroids

- Generating final clustered outputs

Figure 6.3 shows a sample Reducer workflow.



Figure 6.3: Sample Reducer Workflow

6.6.4 Selection of Centroids

Centroid selection plays an important role in determining the efficiency and accuracy of clustering algorithms. This section explains the methods used for selecting and updating centroids during execution.

The discussion may include:

- Random centroid initialization
- Distance-based centroid selection
- Iterative centroid updates
- Convergence conditions

6.7 Summary

This chapter presented the implementation details of the proposed system, including the programming languages, platforms, coding standards, and datasets used during development. The challenges encountered and the strategies adopted to overcome them were discussed. The chapter also explained the algorithms implemented within the system, including the K-Means algorithm and its Mapper and Reducer phases. These implementation details form the basis for the testing and result analysis discussed in the subsequent chapters.



Chapter 7

Software Testing

CHAPTER 7

SOFTWARE TESTING

Software testing is an essential phase in the software development lifecycle that ensures the correctness, reliability, performance, and stability of the proposed system. Testing helps identify defects, validate system functionality, and verify whether the implemented modules satisfy the specified requirements. This chapter presents the testing procedures carried out on the proposed system, including Hadoop cluster testing and system-level testing. Various test cases performed on different modules and functionalities are also discussed in detail.

7.1 Hadoop Cluster Testing

This section discusses the testing performed on the Hadoop cluster environment used for executing the distributed application. The testing ensures proper configuration, communication, and execution of tasks within the Hadoop ecosystem.

The testing process may include:

- Hadoop installation verification
- Cluster configuration validation
- HDFS functionality testing
- MapReduce execution testing
- Node communication verification

7.1.1 Test for Single Node

This subsection explains the testing carried out on a single-node Hadoop cluster setup. The objective of this test is to verify whether the Hadoop services and distributed processing functionalities are operating correctly on a standalone environment.

The testing may include:

- Namenode and Datanode verification
- HDFS read/write operations
- Execution of sample MapReduce programs

- Resource allocation testing

A sample single-node testing workflow may be included below.



Figure 7.1: Sample Single Node Hadoop Testing

7.1.2 Test for Multi Node

This subsection describes the testing performed on a multi-node Hadoop cluster environment. The purpose of this test is to verify distributed processing, node communication, and cluster scalability.

The testing may include:

- Inter-node communication testing
- Distributed data storage verification
- Parallel execution validation
- Fault tolerance testing
- Cluster performance evaluation

Figure 7.2 shows a sample multi-node cluster testing architecture.

7.2 System Testing

System testing verifies the overall functionality and integration of all modules within the proposed system. The testing process ensures that the application behaves as expected under different conditions and validates the outputs generated by the implemented algorithms.



Figure 7.2: Sample Multi Node Hadoop Testing

7.2.1 Test for Mapper

This subsection explains the testing carried out for the Mapper module. The Mapper functionality is verified to ensure proper processing of input data and generation of intermediate key-value pairs.

The Mapper testing may include:

- Input data validation
- Correct cluster assignment
- Intermediate output generation
- Performance verification

A sample Mapper test case table may be included.

Table 7.1: Sample Mapper Test Case

Test Case	Input	Expected Output
Mapper Execution	Sample Dataset	Intermediate Key-Value Pairs

7.2.2 Test for Reducer

This subsection explains the testing performed on the Reducer module to verify the correctness of aggregation and centroid computation processes.

The Reducer testing may include:

- Aggregation correctness
- Centroid update verification
- Final output validation

- Performance testing

7.2.3 Test for Input Data Storage

This subsection discusses the testing of input data storage mechanisms used in the proposed system. The testing ensures that input data is stored correctly and can be accessed efficiently during execution.

The testing may include:

- HDFS storage validation
- Database insertion testing
- File integrity verification
- Data retrieval testing

7.2.4 Test for Output Data Storage

This subsection explains the testing carried out for output data storage and retrieval processes. The testing ensures that processed outputs are correctly stored and can be accessed without data loss.

The testing may include:

- Output file generation
- Database storage validation
- Output retrieval verification
- Data consistency testing

7.2.5 Test for Output

This subsection discusses the validation of the final outputs generated by the proposed system. The generated results are verified against expected outcomes to ensure correctness and reliability.

The output testing may include:

- Accuracy verification
- Performance evaluation

- Visualization testing
- Result comparison with expected values

7.2.6 Test for JobTracker

This subsection explains the testing of the JobTracker component responsible for managing and scheduling MapReduce jobs within the Hadoop environment.

The JobTracker testing may include:

- Job scheduling verification
- Task monitoring
- Failure recovery testing
- Resource allocation validation

7.2.7 Test for Number of Mappers

This subsection discusses the testing carried out by varying the number of Mapper tasks during execution. The objective is to analyze system performance, execution time, and scalability under different processing conditions.

The testing may include:

- Execution time analysis
- Performance comparison
- Resource utilization monitoring
- Scalability evaluation

A graph comparing execution time versus number of mappers may also be included.

7.3 Summary

This chapter presented the testing procedures performed on the proposed system, including Hadoop cluster testing and system-level testing. Various tests related to the Mapper, Reducer, data storage, JobTracker, and output validation were discussed in detail. The testing results ensure that the implemented system satisfies the specified functional and performance requirements while maintaining reliability and scalability.



Figure 7.3: Sample Performance Analysis for Number of Mappers



Chapter 8

Experimental Results

CHAPTER 8

EXPERIMENTAL RESULTS

Experimental results provide a comprehensive evaluation of the proposed system by analyzing its performance, accuracy, efficiency, and reliability under different testing conditions. This chapter presents the results obtained during experimentation and discusses the effectiveness of the implemented methodologies and algorithms. Detailed analysis and evaluation metrics are also included to validate the performance of the proposed system. The observations presented in this chapter help in assessing the overall success and practical applicability of the project.

8.1 Results Analysis

This section presents the outputs and results obtained from the implementation and testing phases of the project. The generated results should be analyzed systematically to verify whether the objectives of the project have been successfully achieved.

The analysis may include:

- Output screenshots
- Graphical representations
- Comparison with expected results
- Performance observations
- Accuracy measurements

Sample output figures or graphs may be included as shown below.



Figure 8.1: Sample Result Analysis

8.2 Detailed Analysis

This section provides an in-depth evaluation of the proposed system using various analytical and performance-based measures. The effectiveness of the algorithms and implementation strategies can be discussed in detail.

The detailed analysis may include:

- Execution time comparison
- Resource utilization
- Accuracy and efficiency evaluation
- Scalability analysis
- Comparative study with existing systems

8.2.1 Performance Analysis

Performance analysis evaluates the efficiency and responsiveness of the proposed system under different operational conditions. The analysis helps determine the computational effectiveness and scalability of the implemented solution.

The performance evaluation may include:

- Processing time analysis
- Memory utilization
- CPU and resource consumption
- Throughput and latency measurements
- Scalability testing

Performance graphs or comparison charts may also be included.

8.2.2 Evaluation Metric

Evaluation metrics are used to quantitatively assess the accuracy and effectiveness of the proposed system. The choice of metrics depends on the application domain and algorithms implemented in the project.

Common evaluation metrics may include:



Figure 8.2: Sample Performance Analysis

- Accuracy
- Precision
- Recall
- F1-Score
- Mean Squared Error (MSE)
- Root Mean Square Error (RMSE)
- Silhouette Score (for clustering applications)

A sample evaluation metrics table may be included below.

Table 8.1: Sample Evaluation Metrics

Metric	Value
Accuracy	95%
Precision	93%
Recall	92%
F1-Score	92.5%

8.3 Summary

This chapter presented the experimental results and detailed analysis of the proposed system. The outputs obtained from implementation and testing were analyzed using performance measures and evaluation metrics. The results demonstrate the effectiveness, efficiency, and reliability of the proposed solution under different operational conditions.



Chapter 9

Conclusion

CHAPTER 9

CONCLUSION

The conclusion summarizes the overall work carried out in the project and highlights the major achievements, findings, and outcomes obtained during the development and evaluation phases. This chapter discusses the effectiveness of the proposed system in addressing the identified problem statement and achieving the specified objectives. The limitations of the current implementation and possible future enhancements are also presented to provide direction for further research and development.

9.1 Limitations

Although the proposed system demonstrates effective performance and functionality, certain limitations may still exist due to technical, operational, or resource-related constraints. This section discusses the boundaries and restrictions of the current implementation.

The limitations may include:

- Limited dataset availability or size
- Hardware and computational constraints
- Dependency on internet connectivity or external services
- Scalability limitations for very large-scale deployments
- Reduced accuracy under specific conditions
- Security and privacy considerations

The discussion should clearly specify the conditions under which the proposed system performs effectively and the scenarios where improvements may be required.

9.2 Future Enhancement

This section discusses the possible improvements and additional functionalities that can be incorporated into the system in future versions. Future enhancements help improve scalability, performance, usability, and adaptability of the proposed solution.

Potential future enhancements may include:

- Integration with cloud computing platforms
- Real-time data processing and analytics
- Mobile application support
- Enhanced security and authentication mechanisms
- Artificial Intelligence (AI) and Machine Learning (ML) based automation
- Improved scalability and distributed processing capabilities
- User interface and visualization improvements

Future work may also focus on optimizing algorithms, improving system efficiency, and expanding the application scope of the project.

The successful implementation and evaluation of the proposed system demonstrate its practical applicability and effectiveness in addressing the identified problem domain. The methodologies, architectures, and technologies adopted during the project provide a strong foundation for future research and further development in this area.



Appendix A

Plagiarism report

APPENDIX A

PLAGIARISM REPORT

Attach the screenshot of plagiarism report front page.





Appendix B

Publication details

APPENDIX B

PUBLICATION DETAILS

Attach the screenshot of either the Acknowledgment of the publication / Proof of Acceptance / Proof of Registration.





Appendix C

Code

APPENDIX C

CODE

C.1 First Appendix

You can use `tcbllisting` for creating the code snippets. The following example illustrates how one can customize the `tcbllisting` to achieve the `tcl` script. Similarly, one can use it for other programming language listing, including HDL.

```
# Since our design has a clock with name clk,
## specify that name under [get_port ]
create_clock -period 40 -waveform {0 20} [get_ports clk]

# Setting a 'delay' on the clock:
set_clock_latency 0.3 clk

# Setting up constraints on your I/P and O/P pins
set_input_delay 2.0 -clock clk [all_inputs]
set_output_delay 1.65 -clock clk [all_outputs]

# Set realistic 'loads' on each output pin
set_load 0.1 [all_outputs]

# Set 'maximum' fanin and fan-out for the input and output pins
set_max_fanout 1 [all_inputs]
set_fanout_load 8 [all_outputs]
```

BIBLIOGRAPHY

- [1] B. Razavi, *Design of Analog CMOS Integrated Circuits*. McGraw-Hill Education, 2000, ISBN: 978-0072380323.

